INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Projet - développement mobile

Vestory

Enzo Martellucci
Baptiste TELLIER

1. Concept, utilisateur cible et objectifs **3**
2. Fonctionnalités prévues et réalisées **4**
Justification des écarts avec le cahier des charges 4
3. Captures des principaux écrans **5**
4. Architecture logicielle **6**
5. Gestion des états **7**
6. Persistance et modèle des données **9**
7. Cache, invalidation et actualisation **10**
8. API et données exploitées **11**
9. Traitements et calculs métier **11**
Graphiques 12
11. Scoring : le calcul de la rareté **14**
12. Tests **15**
13. Difficultés, limitations et améliorations **15**
Annexe **15**

1. Concept, utilisateur cible et objectifs

Pour un débutant, l'investissement paraît compliqué et les outils existants sont soit des plateformes de trading intimidantes, soit des cours théoriques peu motivants. Vestory rend cette découverte aussi motivante qu'un jeu de cartes à collectionner, puis fait vivre au joueur, avec de vrais cours, ce que signifie investir sur plusieurs années. L'application repose sur deux boucles de jeu :

- **Collectionner** : chaque carte est un vrai actif (action, ETF, cryptomonnaie, paire de devises, matière première). Le joueur ouvre des packs (un gratuit par jour, d'autres achetés en gemmes), découvre la rareté, le type, le pays et le secteur de l'actif, puis consulte sa fiche : description, cours du jour, historique.
- **Investir dans le passé** : une partie transporte le joueur à un mois tiré au hasard entre 2005 et il y a 5 ans, avec 1 000 $ fictifs. Il investit uniquement dans les actifs de sa collection, au vrai cours historique du mois. Chaque jour réel fait avancer le marché d'un mois, pendant 60 mois.

Pourquoi ce « voyage dans le temps » ? Avec des cours en temps réel, un portefeuille fictif bouge très peu en quelques semaines et l'utilisateur n'apprend rien. En rejouant 5 ans d'historique réel en 60 jours, il traverse de vraies périodes de marché et voit l'effet du temps, de la diversification et des baisses sur sa courbe de valeur.

Pourquoi des cartes à collectionner ? La curiosité est le premier frein d'un débutant : il ne regarde que les quelques entreprises qu'il connaît. Le hasard des packs lui fait découvrir des actifs auxquels il n'aurait jamais pensé, et la rareté, calculée à partir du poids réel de l'actif sur son marché, lui apprend implicitement qui sont les géants de chaque secteur. Investir uniquement dans ses cartes relie les deux boucles : collectionner donne plus de choix pour diversifier.

Utilisateur cible : une personne novice, habituée aux jeux mobiles, qui veut comprendre l'investissement sans risquer d'argent. **Objectifs :** donner envie de découvrir des actifs variés, faire apprendre par l'interaction avec des données réelles, rester utilisable hors connexion et proposer une application bien structurée et testée. Aucune transaction réelle n'est effectuée ; gemmes et solde sont fictifs.

2. Fonctionnalités prévues et réalisées

Fonctionnalité (cahier des charges)	État	Commentaire
Profil joueur, portefeuille fictif de 1 000	Réalisée	Pseudo validé par formulaire ; 1 000 $ par partie (cours en dollars).
Pack quotidien gratuit, boutique à 3 packs	Réalisée	Standard 200, Premium 1 500, Elite 10 000 gemmes, probabilités affichées.
Doublons convertis en gemmes	Réalisée	2 / 10 / 50 / 100 gemmes selon la rareté.
Collection et fiche d'un actif	Réalisée	Recherche, tri, cartes verrouillées, carte retournable, cours et historique.
Achat / vente par fractions, suivi du portefeuille	Réalisée	Positions, plus-values, courbe de valeur, historique des ordres, parties passées.
3 packs de bienvenue	Modifiée	Remplacés par 2 100 gemmes de départ : le joueur choisit ses packs.
3 cartes par pack	Modifiée	1 carte par pack, car le catalogue ne compte que 200 actifs pour l'instant.
Jeu « voyage dans le temps »	Ajoutée	Remplace un portefeuille en temps réel, jugé peu pédagogique.
Quiz quotidiens, score et niveau	Non réalisée	Manque de temps ; seules des statistiques de profil existent.

Les contraintes du sujet sont couvertes : 7 écrans avec navigation, formulaires validés, listes dynamiques, API REST convertie en objets Dart, persistance métier, cache local, mode hors connexion, états de chargement / erreur, graphiques, calculs métier, animations.

Justification des écarts avec le cahier des charges

- **Gemmes de départ plutôt que 3 packs offerts** : le joueur découvre tout de suite la boutique et les probabilités de chaque pack, et fait un premier choix (1 Premium ou 10 Standard), ce qui est plus engageant qu'un cadeau imposé.
- **1 carte par pack plutôt que 3** : le catalogue ne compte pour l'instant que 200 actifs. Avec 3 cartes par pack, la collection se compléterait très vite et les doublons s'accumuleraient, ce qui enlèverait l'intérêt de l'ouverture. Avec une seule carte, la progression est plus lente et chaque ouverture est mise en scène (indices sur le type, le pays et le secteur). Le passage à 3 cartes par pack sera possible lorsque le catalogue sera enrichi.
- **Jeu dans le passé plutôt que portefeuille en temps réel** : c'est le choix qui sert le mieux l'objectif pédagogique, et il exploite les historiques que le sujet suggère d'utiliser.
- **Quiz non réalisés** : ils n'étaient pas prioritaires par rapport au cœur de l'application (packs, collection, jeu d'investissement, cache et mode hors connexion), et nous avons manqué de temps pour les développer.

3. Captures des principaux écrans

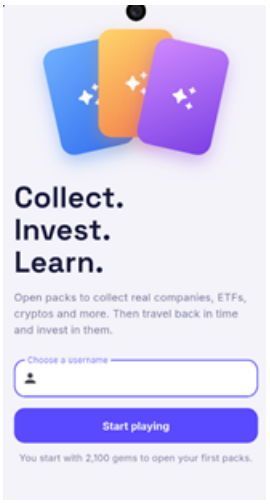


Figure a – Onboarding

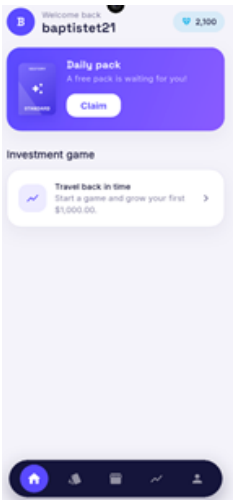


Figure b – Accueil

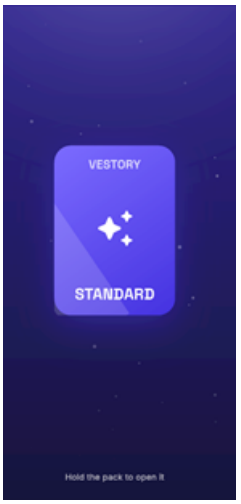


Figure c – Ouverture d'un pack

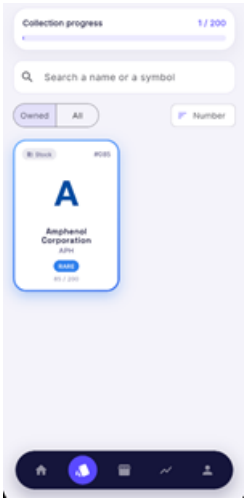


Figure d – Collection

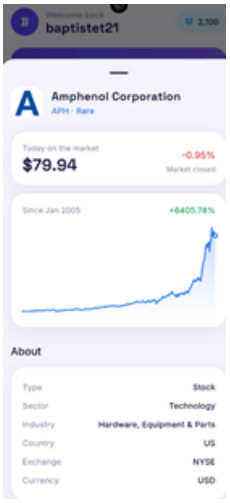


Figure e – Fiche d'un actif

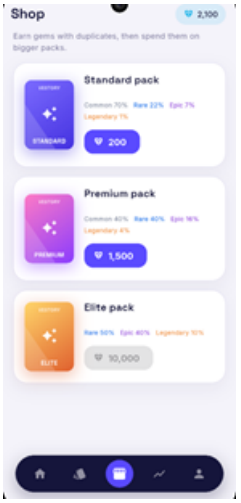


Figure f – Boutique

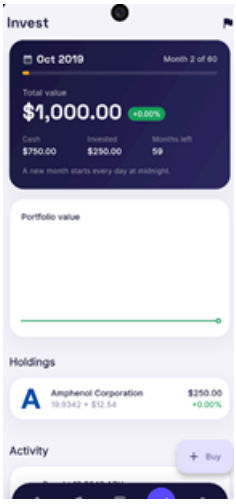


Figure g – Invest : partie en cours

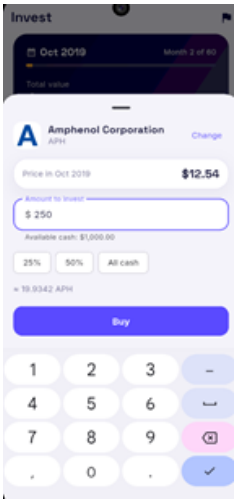


Figure h – Formulaire

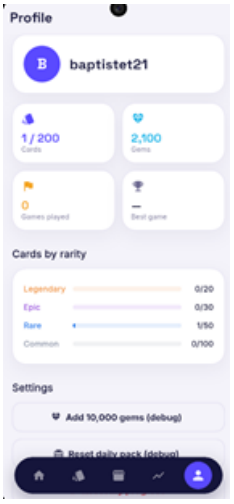


Figure i – Profil

4. Architecture logicielle

Choix initial puis abandonné : nous étions partis sur un backend Spring Boot (PostgreSQL, authentification JWT) qui gérait comptes, packs et portefeuille et appelait lui-même les API externes. Il a été supprimé en fin de projet : Vestory est désormais uniquement une application Flutter avec des bases SQLite locales. **Justification** : le sujet met l'accent sur la persistance locale et le hors connexion (avec un serveur, ouvrir un pack ou acheter devenait impossible sans réseau) ; un seul langage et un seul modèle de données simplifient le projet et la démonstration ; les refactorings du backend créaient de nombreux conflits.

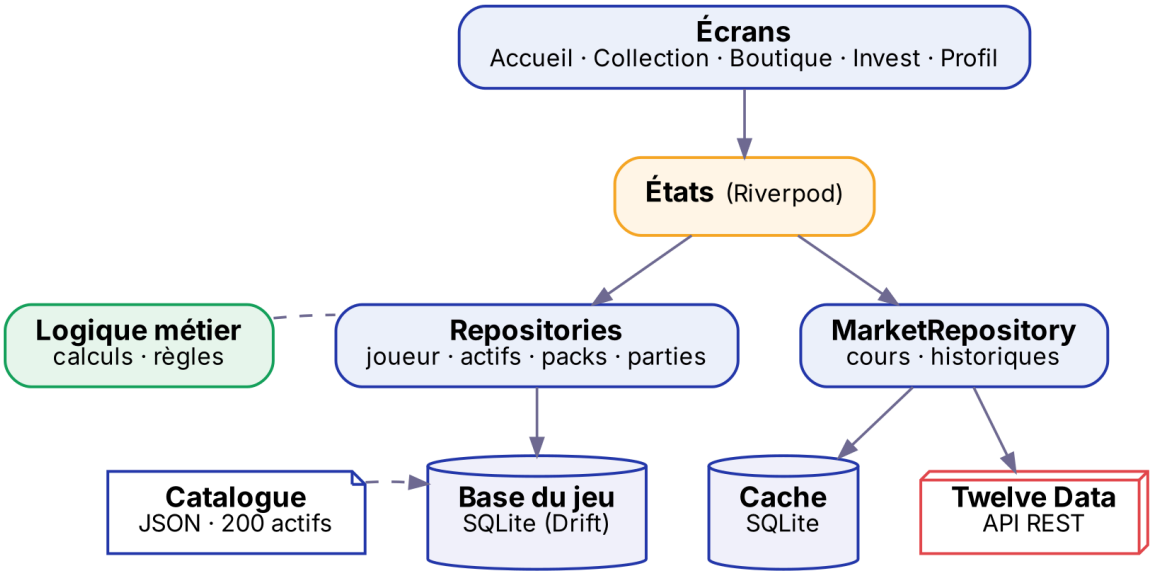


Figure 1 – Principaux composants et couches

Le code est organisé **par fonctionnalité** (player, assets, packs, collection, shop, market, game), chacune découpée en data/ (repositories), domain/ (règles en Dart pur) et presentation/ (écrans, providers). Le dossier core/ contient la base, le client réseau, le routeur et les composants partagés. Un écran ne parle jamais directement à la base ni au réseau : il observe un provider, qui s'appuie sur un repository, qui délègue les règles au domaine.

Choix	Alternative écartée	Justification
Application Flutter seule, bases locales	Backend Spring Boot + PostgreSQL	Hors connexion natif, aucune infrastructure à lancer, un seul modèle de données. Le serveur n'apportait rien d'indispensable puisqu'il n'y a pas de données partagées entre joueurs.
Organisation par fonctionnalité	Organisation par couche (tous les écrans ensemble, tous les repositories ensemble)	Tout ce qui concerne les packs ou le jeu est au même endroit ; on travaille à deux sur des dossiers différents, ce qui limite les conflits.

Logique métier en Dart pur (domain/)	Calculs dans les widgets ou les repositories	Testable sans Flutter ni base ; la même règle sert au formulaire et à l'enregistrement.
go_router avec redirections		Les redirections centralisent la logique « joueur créé ou non » ; StatefulShellRoute conserve l'état de chaque onglet.

5. Gestion des états

Nous avons choisi **Riverpod 3**. Le joueur, la collection et la partie sont partagés entre plusieurs onglets, ce que setState ne permet pas proprement ; Riverpod gère nativement l'asynchrone, combine des providers et permet de les remplacer dans les tests.

Solution	Pourquoi nous ne l'avons pas retenue / retenue
setState seul	Suffisant pour un état local, mais le joueur, la collection et la partie sont lus par 4 onglets : il aurait fallu les remonter et les passer de widget en widget.
Provider	Proche dans l'esprit, mais moins adapté à l'asynchrone et aux dépendances entre états.
Riverpod 3 (retenu)	AsyncValue pour chargement / données / erreur, providers qui se combinent (le portefeuille dépend de la partie, des ordres et des cours), family pour un état par actif, remplacement simple dans les tests.

Type d'état	Mise en œuvre et justification
Données locales	StreamProvider sur les requêtes watch() de Drift : après un achat ou un pack, la base change et tous les écrans se mettent à jour sans invalidation manuelle.
Données distantes	FutureProvider.family par actif (historique, cours).
Données dérivées	portfolioProvider combine partie, ordres, historiques et horloge, puis appelle le domaine.
Navigation	Le routeur observe le joueur : chargement, création du profil, ou accueil.
États locaux	setState pour la recherche, le tri, les formulaires ; AnimationController et machine à états OpeningPhase pour l'ouverture des packs.

Pourquoi des flux Drift plutôt que des invalidations manuelles ? Avec un FutureProvider, il faudrait penser à invalider le profil, la collection et le portefeuille après chaque action ; un oubli donne un écran périmé. En observant la base, la source de vérité notifie elle-même les écrans. **Pourquoi une horloge injectable ?** Le jeu dépend du jour courant (pack quotidien, mois de marché) : sans elle, ces règles seraient impossibles à tester. **Pourquoi une machine à états pour l'ouverture ?** Une énumération OpeningPhase rend chaque étape explicite, empêche de quitter l'écran pendant l'animation et garantit que la carte est tirée en base avant l'explosion.

6. Persistance et modèle des données

Les **données du joueur** sont séparées des **données Internet** (simple copie en cache), dans deux bases SQLite distinctes. Nous avons choisi **Drift** : base relationnelle typée, transactions, clés étrangères et surtout requêtes observables qui s'intègrent aux StreamProvider. Un stockage clé-valeur n'aurait pas permis de représenter proprement les ordres ni de rendre un achat atomique.

Solution	Analyse
shared_preferences	Clé-valeur : adapté aux préférences, pas à des listes d'ordres liées à des parties et des actifs.
sqflite (SQL brut)	Même moteur SQLite, mais requêtes en chaînes de caractères, sans typage ni flux réactifs.
Drift (retenu)	SQLite typé et généré, transactions, clés étrangères, énumérations, requêtes watch() reliées à Riverpod, base en mémoire pour les tests.

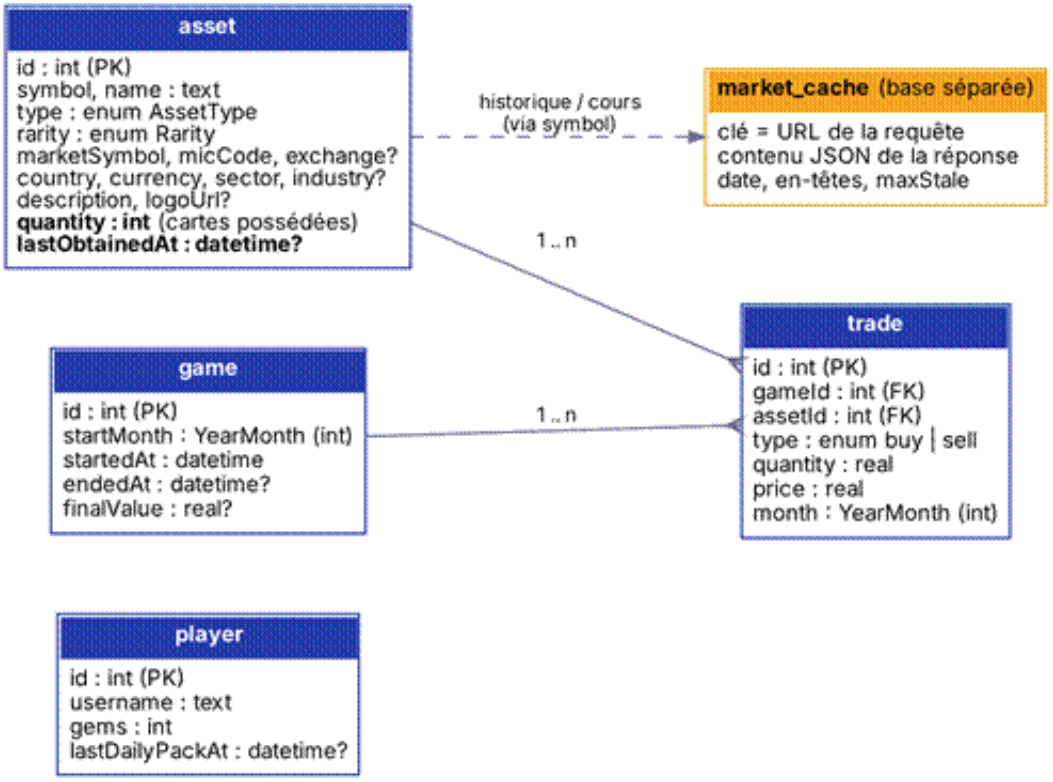


Figure 2 – Modèle des données persistées

Le catalogue de 200 actifs est livré en JSON et copié dans la table asset au premier lancement ; quantity indique les exemplaires possédés. **Les positions et le solde ne sont pas stockés** : ils sont recalculés en rejouant les ordres. Aucune incohérence n'est donc possible, et le portefeuille peut être reconstitué à n'importe quel mois pour tracer la courbe. Ouvrir un pack, acheter ou vendre sont des transactions.

Pourquoi deux bases séparées ? Le cache peut être vidé ou corrompu sans aucun risque pour la progression du joueur, et la base de jeu reste petite et lisible. **Pourquoi le catalogue en JSON ?** Il ne change pas, il est disponible dès le premier lancement sans réseau, et il ne consomme aucun appel API. **Pourquoi rejouer les ordres ?** Stocker un solde et des positions obligerait à les maintenir à jour à chaque ordre.

7. Cache, invalidation et actualisation

Les réponses de Twelve Data sont conservées par un **cache HTTP persistant** (dio_cache_interceptor + base SQLite market_cache), indexé par l'URL de la requête. La politique forceCache sert la réponse locale tant qu'elle est valide, sans appel réseau.

Pourquoi un cache HTTP ? L'intercepteur se place entre Dio et le réseau : les repositories ne changent pas, la clé de cache (l'URL complète) distingue automatiquement chaque actif et chaque paramètre, et la date de stockage est gérée pour nous. Une table maison aurait demandé de réécrire sérialisation, expiration et repli hors ligne.

Donnée	Validité	Justification
Historique mensuel d'un actif	Illimitée	Le jeu n'utilise que des mois passés, qui ne changent plus : un actif n'est téléchargé qu'une fois, puis lu depuis le disque, même après redémarrage.
Cours du jour	12 heures	Information d'affichage ; deux actualisations par jour suffisent et protègent le quota gratuit.
Catalogue	Livré avec l'app	Ne change pas : aucun appel réseau.

Pourquoi des clôtures mensuelles ? Un mois de jeu dure un jour réel : une granularité plus fine serait invisible pour le joueur. **Pourquoi forceCache plutôt que « réseau d'abord » ?** Pour des données passées, interroger le réseau n'apporte rien et consomme le quota.

Invalidation : une entrée expirée est remplacée à la récupération suivante, les réponses renvoyées avec un code d'erreur HTTP ne sont jamais mises en cache, si le réseau échoue, l'ancienne réponse est servie plutôt qu'une erreur, sauf pour un cours du jour expiré, supprimé du cache au redémarrage. **Hors connexion**, tout le jeu reste utilisable (packs, collection, parties, achats sur les actifs déjà chargés) puisque les données du joueur sont locales, un actif jamais consulté affiche un message clair et un bouton pour réessayer.

8. API et données exploitées

- **Pendant l'utilisation – Twelve Data** : /time_series (clôtures mensuelles depuis 2005, une requête par actif) pour le jeu et les graphiques, et /quote (cours, variation, marché ouvert) pour la fiche. Le JSON est converti en objets Dart, les erreurs deviennent des messages lisibles.
- **En amont – FMP et Twelve Data** : appelées une seule fois, à l'époque par le backend, pour récupérer les 200 actifs (pays, secteur, description, logo, capitalisation, volume), calculer leur rareté et produire les fichiers JSON du catalogue.

Pourquoi Twelve Data ? Une seule API couvre les cinq types d'actifs du catalogue, accepte un code de marché pour lever les ambiguïtés de symbole et renvoie l'historique mensuel sur 20 ans en un appel. En offre gratuite, FMP ne fournit l'historique des cours que pour 71 de nos 200 actifs, le reste étant réservé aux offres payantes.

Justification du découpage : il évite de consommer le quota gratuit (800 crédits / jour) pour des informations qui ne changent pas et rend la collection disponible hors ligne.

9. Traitements et calculs métier

Les calculs sont regroupés dans le dossier domain/, séparé de l'interface. Les mêmes fonctions servent à vérifier le formulaire d'achat ou de vente et à enregistrer l'ordre, donc la règle est la même partout.

Calcul	Principe	Exemple
Avancement de la partie	Chaque jour réel, la partie avance d'un mois de marché.	Partie commencée en janvier 2015 : le lendemain, on est en février 2015.
Quantité achetée	Montant investi ÷ cours de l'actif. L'achat est refusé si le montant est nul, dépasse l'argent disponible ou si l'actif n'existait pas encore à cette date.	100 $ sur une action à 40 $ donnent 2,5 actions.
Prix d'achat moyen	Moyenne des prix payés pour un actif acheté en plusieurs fois.	2 actions à 10 $ puis 2 à 20 $ font un prix moyen de 15 $.
Gain ou perte	Valeur actuelle − montant investi, aussi en pourcentage.	100 $ investis qui valent 125 $ font un gain de +25 $ (+25 %).
Valeur du portefeuille	Argent non investi + valeur de toutes les positions.	900 $ non investis + 125 $ d'actions = 1 025 $.
Performance de la partie	Comparaison de la valeur finale avec les 1 000 $ de départ.	Fin à 1 180 $: +18 %.
Tirage d'un pack	Tirage au sort de la rareté selon les probabilités du pack.	Pack Standard : 70 % commune, 22 % rare, 7 % épique, 1 % légendaire.

Choix des règles :

- **1 jour réel = 1 mois de marché.** Une partie de 5 ans dure ainsi environ 2 mois.
- **Mois de départ tiré au hasard.** Le joueur ne sait pas à l'avance ce qui va se passer, et chaque partie est différente.
- **Un actif ne peut pas être acheté avant d'exister.** Par exemple, on ne peut pas acheter de Bitcoin en 2006. Cela garde la partie réaliste.

Graphiques

L'application trace deux courbes, avec un point par mois :

- l'historique d'un actif, dans sa fiche : les cours mensuels depuis 2005, avec l'évolution totale sur la période ;
- la valeur du portefeuille, dans l'écran Invest : la valeur totale (argent non investi et positions) à chaque mois écoulé de la partie, avec une ligne pointillée à 1 000 $ pour voir immédiatement le gain ou la perte.

Les deux utilisent un seul composant, LineChart, dessiné avec un CustomPainter. Il trace la courbe, une zone en dégradé dessous et un point sur la dernière valeur. La courbe se dessine progressivement à l'affichage grâce à un TweenAnimationBuilder, et celle du portefeuille est verte ou rouge selon la performance.

Pourquoi un graphique fait main plutôt qu'une bibliothèque ? Le besoin est simple : une seule courbe, sans axes interactifs. Un CustomPainter donne un contrôle total sur le style, n'ajoute aucune dépendance, et l'animation du tracé fait partie des animations codées.

D'où viennent les points du portefeuille ? Le solde et les positions ne sont pas stockés : la courbe est obtenue en rejouant les ordres mois par mois. Ce calcul est couvert par les tests unitaires.

10. Animations

Toutes les animations sont codées sans package externe, et chacune sert l'expérience :

Animation	Intérêt	Implémentation
Ouverture du pack (appui long, tremblement, charge, explosion)	Créer du suspense et rendre l'ouverture plus marquante	AnimationController, AnimatedBuilder, CustomPainter
Séquence d'indices puis révélation de la carte	Faire découvrir l'actif avant de le montrer	AnimatedSwitcher + Timer ; Interval et rotation Matrix4
Particules et rayons selon la rareté	Marquer la rareté	CustomPainter piloté par AnimationController
Retournement des cartes	Passer de la carte à ses infos	AnimationController, Transform rotateY
Tracé des courbes, compteurs animés	Rendre les variations visibles	TweenAnimationBuilder, Tween sur les valeurs

Pourquoi aucun package externe ? Le sujet demande des animations programmées par le groupe, et on voulait qu'elles changent selon la carte obtenue. Plus la carte est rare, plus l'ouverture du pack est spectaculaire : l'attente est plus longue, il y a plus de particules et un halo plus lumineux. Le téléphone vibre aussi au moment où le pack s'ouvre. Pour que l'application reste fluide, les effets de particules sont dessinés directement à l'écran, en un seul dessin plutôt qu'avec un élément par particule. Chaque animation est aussi arrêtée quand on quitte l'écran, pour ne pas consommer de ressources inutilement.

11. Scoring : le calcul de la rareté

La rareté d'une carte dépend de l'importance réelle de l'actif sur son marché. Une carte légendaire correspond donc à un « géant » de sa catégorie, ce qui apprend au joueur qui sont les acteurs majeurs.

Comment le score est calculé. Chaque actif reçoit une note de 0 à 100. Il n'est comparé qu'aux actifs du même type : les actions avec les actions, les cryptos avec les cryptos, etc.

- **Actions, ETF et cryptomonnaies :** la note dépend surtout de la taille de l'entreprise ou de l'actif (sa capitalisation, pour 80 %) et un peu du volume échangé chaque jour (pour 20 %).
- **Matières premières :** la note dépend uniquement du volume échangé.
- **Devises :** la note dépend de la popularité de la paire. Les paires les plus échangées, comme EUR/USD, ont 100 et les plus rares ont 40.

On regarde le rang de l'actif dans sa catégorie plutôt que sa taille brute, car les écarts sont énormes : Apple vaut des milliers de fois plus qu'une petite entreprise. Ensuite, tous les actifs sont classés : les 10 % meilleurs sont légendaires, les 15 % suivants épiques, les 25 % suivants rares et les 50 % restants communs.

Un calcul fait une seule fois. Le calcul demandait environ 200 appels aux API. Le refaire à chaque lancement aurait épuisé notre quota gratuit et changé la rareté de cartes que le joueur possède déjà. Il a donc été lancé une seule fois, et le résultat est enregistré dans le catalogue de l'application.

Aide de l'IA. Nous avons utilisé un assistant d'intelligence artificielle pour concevoir cette méthode de calcul.

Ce score est un outil pédagogique basé sur des données passées. Ce n'est pas un conseil d'investissement.

12. Tests

Nous avons d'abord testé ce qui peut fausser l'expérience sans se voir (calculs du portefeuille, règles de temps, tirage, cache), puis les interactions délicates (appui long, séquence passable). Les tests sont ainsi rapides, reproductibles et indépendants de l'API.

Domaine testé	Cas limites couverts
Achat, vente, positions, liquidités, courbe	Montant nul, liquidités insuffisantes, vente supérieure à la détention, cours absent
Horloge et règles de partie	Fin à 60 mois, partie terminée figée, aucune partie jouée
Tirage des packs, pack quotidien	Rareté absente, catalogue vide, deux ouvertures le même jour, passage de minuit
Cache du marché	Historique téléchargé une fois, redémarrage, hors ligne, erreurs non mises en cache, expiration
Repositories, validation du pseudo, widgets	Pack trop cher sans effet, achat hors collection, pseudo invalide, appui relâché trop tôt

55 tests au total, lancés avec flutter test dans mobile/, dont des tests d'intégration sur une vraie base SQLite en mémoire.

13. Difficultés, limitations et améliorations

Difficultés : le changement d'architecture en cours de projet (backend puis réécriture de la couche de données autour de Drift, conflits de fusion) ; les quotas gratuits des API, qui nous ont poussés à figer le catalogue et à télécharger l'historique une seule fois ; l'identification des actifs selon leur place de cotation, la conception d'un temps de jeu à la fois jouable et testable.

Améliorations possibles, non réalisées faute de temps :

- ajouter les **quiz quotidiens** prévus au cahier des charges.

Bilan : un débutant peut découvrir de vrais actifs par le jeu puis investir fictivement sur de vraies années de marché. L'architecture en couches permet d'ajouter ces évolutions sans remettre en cause l'existant.

Annexe

1. Installer Flutter, puis ouvrir le dossier `mobile/`.
2. Copier `env.example.json` en `env.json` et y mettre une clé API Twelve Data (offre gratuite).
3. Installer les dépendances : `flutter pub get`.
4. Lancer l'application : `flutter run --dart-define-from-file=env.json`.
5. Lancer les tests : `flutter test`.